新プロジェクト概要：神事特化 LLM 構築・検証（4 月～6 月）

2026/3/31
佐野

本プロジェクトは、C 言語および独自ミドルウェア「Moove」環境において、AI エージェントを駆使して「レビュー」と「テスト生成」の限界に挑む実証実験です。

1. プロジェクトの全体像と目的

(1) ソースコードレビューの高度化検証

背景：既存の静的解析ツール（Coverity）は、C 言語の一般的な不備は検知できますが、Moove 特有の作法（関数呼び出し順序、リソース解放のタイミング等）は検知できません。

目的：AI に Moove のドメイン知識を注入し、「Coverity の死角」をカバーするインテリジェントなレビューを実現します。

ポイント：MESW 提供のレビュー観点（例：「Open したら必ず Close する」といった論理整合性）を、AI がコードから読み取れるようにします。

アプローチ

- ドメイン注入：Moove マニュアルを Markdown 化し、MCP サーバや Agent Skills として AI に読み込ませます。
- エージェント活用：Cline や Codex といった自律型エージェントを使用し、MESW 提供のレビュー観点（例：Open/Close の整合性）に基づいた指摘を行わせます。

(2) 単体試験（Unit Test）自動化の検証

背景：単体試験コードの作成は工数がかかりますが、これを「専用の生成プログラム」を作って自動化すると、仕様変更のたびにそのプログラム自体を直す必要があり、汎用性が失われます。

目的：「プロンプトエンジニアリング」のみで、AI に直接高品質なテストコードを書かせ、メンテナンスコストを最小化します。

ポイント：コードの特徴に合わせて、AI に「何を与え、どう指示すれば、追加の実装なしで最適なテストが書けるか」のノウハウを確立します。

検証フロー：

- 生成と実行：AI が生成したテストコードを、実際に Moove 実行環境でビルド・実行し、期待通りのテスト結果が得られるかまでを一貫して検証します。
- 指示の最適化：ソースコードの特徴（複雑度や依存関係）に応じ、AI にどのようなコンテキスト（情報）を与えれば最高の結果が出るかの「指示の法則」を確立します。

(3) 【共通方針】

- 脱・独自プログラム：AI を補助する周辺プログラムを自作するのではなく、Cline、Codex、GitHub Copilot といった既存のエージェントツールを使い倒す。
- 知識の外部化：ツールが知らない「Moove」の知識は、MCP サーバや Agent Skills として外部から注入する。

2. チーム体制と進め方の検討素材

3 名（プラディオット、クマリ、ソニ）でこれらを進めるにあたり、どのような体制が効率的か、以下の観点から話し合って決めてください。

担当の分け方

- **機能別（A/B チーム制）：**「レビュー担当」と「テスト生成担当」に分かれ、それぞれの専門性を深める。
- **工程別（全方位制）：**3 人全員が両方の課題に触れ、知見を共有しながら進める。

スケジュールの意識：4 月 1 日〜6 月 29 日の 3 ヶ月間で、「アイデア出し」→「プロトタイプ作成」→「実証・評価」のサイクルを回す必要があります。

3. 次週までの検討課題

次週の MTG（4 月 7 日 16 時）では、2 つの課題に対し、「どのようなアプローチ（戦略）」で臨み、「具体的にどのようなアイデア」を試すべきかを議論します。各自、以下の視点を参考に、自身の担当したい領域について可能な限り多くの提案を準備してください。

A) アプローチの多角的な検討

「AI を賢く動かす」ためには、プロンプト以外にも多くの手段があります。以下のような切り口でアイデアを考えてみてください。

知識・ルールの構造化（Input の工夫）

- マニュアルをそのまま渡すのではなく、ルールを細分化・タグ付けして渡す。
- MCP サーバや Agent Skills に、どのような形式で「仕様」を保持させるのが最適か。

検証・評価の仕組み（Process の工夫）

- レビュー指摘が正しいかどうかを、AI 自身にダブルチェックさせるプロセス。
- 「あえてバグを仕込んだコード」を使い、AI の検知能力をスコアリングする。

環境とエージェントの連携（Tool の工夫）

- Cline や Codex に対し、どのような外部ツール（コンパイラや静的解析結果など）を連携させれば、より精度の高い回答が得られるか。

B) プロンプトエンジニアリングによる改善

プロンプトの書き方一つで、AI のアウトプットは劇的に変わります。以下のような手法（例）を含め、どのような指示出しが有効か検討してください。

指示手法の例

- **Few-shot：**理想的なレビュー指摘やテストコードの「手本」を提示し、パターン学習させる。
- **Chain of Thought：**推論のプロセス（手順）を指定し、論理的な思考を経て回答させる。
- **Role Prompting：**特定の専門領域（組み込みセキュリティ、QA エンジニア等）になりきらせる。

「自分たちが AI エージェントの監督（Director）ならどう操るか」という視点で、具体的なアイデアをリストアップし、この PJ をどのように進めていくかの方針をまとめてください。

New Project Overview: Development & Verification of Shinto-Specialized LLM (April – June)

2026/3/31
Chiiho Sano

This project is an experimental initiative to push the boundaries of "Code Review" and "Test Generation" using AI agents within a C language and proprietary middleware "Moove" environment.

1. Project Scope and Objectives

(1) Advanced Source Code Review Verification

Background: Existing static analysis tools (e.g., Coverity) can detect general C language flaws but cannot identify issues related to "Moove-specific conventions" (e.g., function call sequences, resource release timing).
Objective: Inject Moove domain knowledge into the AI to achieve intelligent reviews that cover "Coverity's blind spots."
Key Point: Ensure the AI can interpret logical consistency based on review viewpoints provided by MESW (e.g., "Any 'Open' must always be followed by a 'Close'").
Approaches:

- Domain Injection: Convert Moove manuals into Markdown format and integrate them into the AI via MCP Servers or Agent Skills.
- Agent Utilization: Use autonomous agents such as Cline and Codex to perform reviews based on the specific viewpoints provided by MESW.

(2) Verification of Automated Unit Test Generation

Background: Manual test code creation is labor-intensive. However, automating this with "proprietary generation programs" results in a loss of versatility, as the program itself must be modified whenever specifications change.
Objective: Use "Prompt Engineering" alone to have the AI directly generate high-quality test code, thereby minimizing maintenance costs.
Key Point: Establish know-how on "what information to provide and how to instruct the AI" to generate optimal tests without additional implementation, tailored to the characteristics of the code.
Verification Flow:

- Generation & Execution: Establish an end-to-end flow where AI-generated test code is built and executed in the actual Moove environment to verify that expected results are obtained.
- Instruction Optimization: Establish "Instruction Rules" to determine what context (information) yields the best results based on the source code's characteristics (complexity, dependencies).

(3) 【Common Policies】

- No Proprietary Programs: Avoid creating custom auxiliary programs. Instead, fully utilize existing agent tools such as Cline, Codex, and GitHub Copilot.
- Externalization of Knowledge: Any "Moove" knowledge unknown to the tools should be injected externally via MCP Servers or Agent Skills.

2. Team Structure and Implementation Considerations

The three of you (Pradyot, Kumari, and Soni) should discuss and decide on the most efficient structure based on the following:
Role Allocation:

- Functional (A/B Team): Split into a "Review Team" and a "Test Generation Team" to deepen expertise in each area.
- Phase-based (All-hands): Everyone works on both tasks to share knowledge and insights.

Schedule Awareness: Within the 3-month window (April 1 – June 29), we must cycle through: "Ideation" → "Prototyping" → "Verification & Evaluation."。

3. Key Tasks for Next Week

During the next meeting (April 7, 16:00), we will discuss the "Strategy" and "Specific Ideas" for both tasks. Please prepare as many proposals as possible for the area you wish to lead, using the following viewpoints as a guide:

A) Multidimensional Approach

There are many ways to "drive AI intelligently" beyond just prompting. Consider ideas from these angles:

Input Optimization (Knowledge/Rule Structuring):

- Instead of passing manuals as-is, how can we break down and tag the rules?
-
- What is the optimal format for storing "specifications" in MCP Servers or Agent Skills?

Process Optimization (Verification/Evaluation Framework):

- Processes where the AI double-checks the validity of its own review comments.
-
- Using "bug-injected code" to score the AI's detection capabilities.

Tool Optimization (Environment & Agent Integration):

- Which external tools (compilers, static analysis results, etc.) should be linked with Cline/Codex to improve response accuracy?

B) Improvement through Prompt Engineering

The AI's output changes drastically based on the prompt. Consider what types of instructions are effective, including:

Examples of Instruction Methods:

- Few-shot: Providing "model examples" of ideal reviews or test code for pattern learning.
- Chain of Thought: Specifying the reasoning process (step-by-step) before the AI provides the answer.
- Role Prompting: Having the AI act as a specialist (e.g., Embedded Security or QA Engineer).

Please list specific ideas and summarize your proposed roadmap for the project from the perspective of: "If you were the Directors of these AI agents, how would you command them?"